Assignment 20.3

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Name: Manupati Eshwar

Hall Ticket No: 2303A51495

Task 1 – Password Strength Validation

Prompt

Generate a Python user registration program

AI Generated Code (Insecure)

```
username = input("Enter username: ")

password = input("Enter password: ")


if len(password) > 0:

   print("User registered successfully!")

else:

   print("Invalid password")
```

Issues Identified

- No minimum password length
- Accepts weak passwords like "123"
- No complexity check
- No protection against common passwords

Secure Code (Fix)

```
import re


def is_strong_password(password):
```

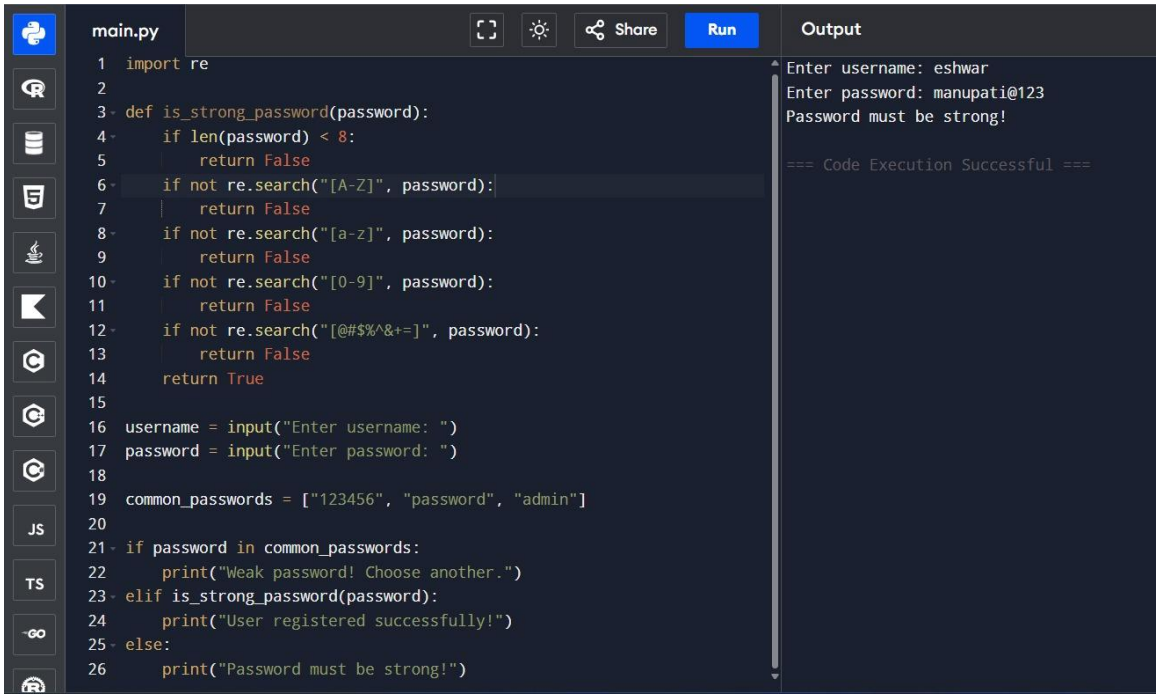
```
    if len(password) < 8:
        return False
    if not re.search("[A-Z]", password):
        return False
    if not re.search("[a-z]", password):
        return False
    if not re.search("[0-9]", password):
        return False
    if not re.search("[@#$%^&+=]", password):
        return False
    return True

username = input("Enter username: ")
password = input("Enter password: ")

common_passwords = ["123456", "password", "admin"]

if password in common_passwords:
    print("Weak password! Choose another.")
elif is_strong_password(password):
    print("User registered successfully!")
else:
    print("Password must be strong!")
```

Explanation

Adds validation rules

Output

Strong accepted, weak rejected



Task 2 – File Upload Security

Prompt

Generate Flask file upload app

AI Generated Code (Insecure)

from flask import Flask, request

app = Flask(__name__)

@app.route('/upload', methods=['POST'])

def upload():

file = request.files['file']

file.save(file.filename)

return "File uploaded"

app.run()

Issues Identified

No validation or size limit

Secure Code (Fix)

```
from flask import Flask, request

import os


app = Flask(__name__)


UPLOAD_FOLDER = 'uploads'

ALLOWED_EXTENSIONS = {'png', 'jpg', 'pdf'}

MAX_FILE_SIZE = 2 * 1024 * 1024  # 2MB


app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER


def allowed_file(filename):

  return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS


@app.route('/upload', methods=['POST'])

def upload():

  file = request.files['file']


  if not allowed_file(file.filename):

    return "Invalid file type"


  if len(file.read()) > MAX_FILE_SIZE:

    return "File too large"
```

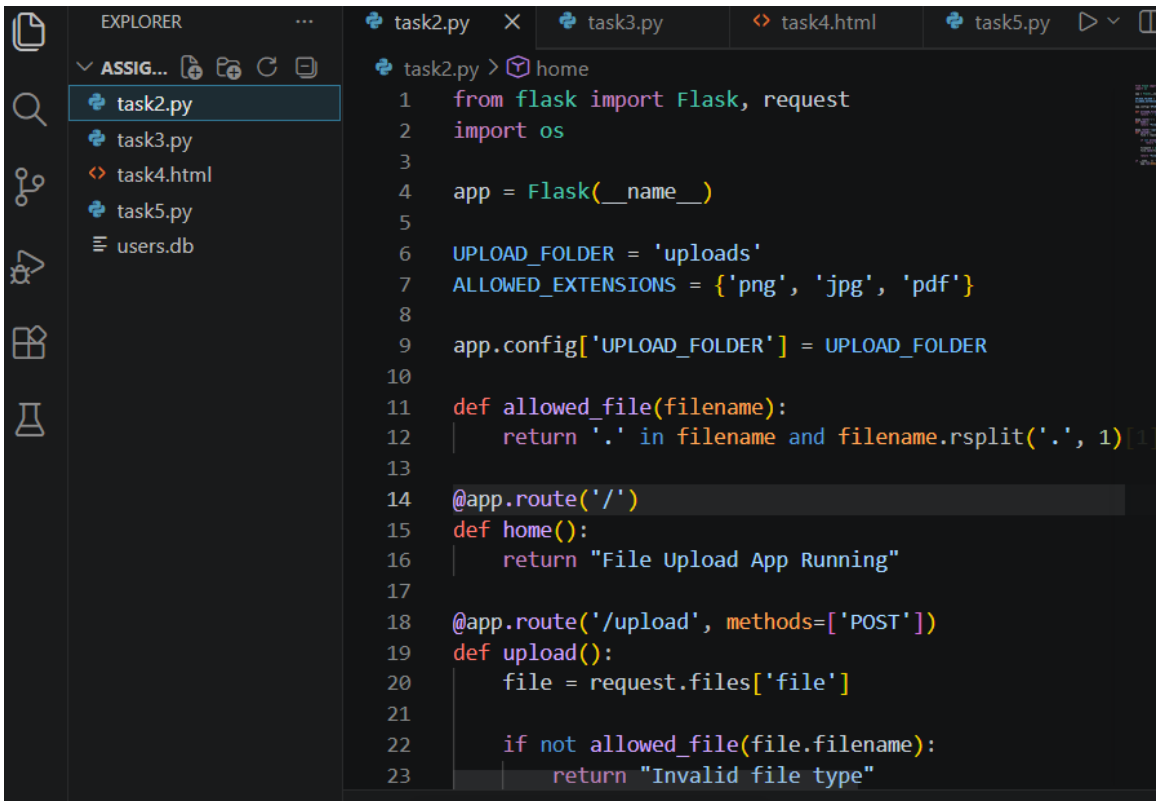
```
    file.seek(0)

    filepath = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)

    file.save(filepath)


    return "Secure upload successful"


app.run()
```

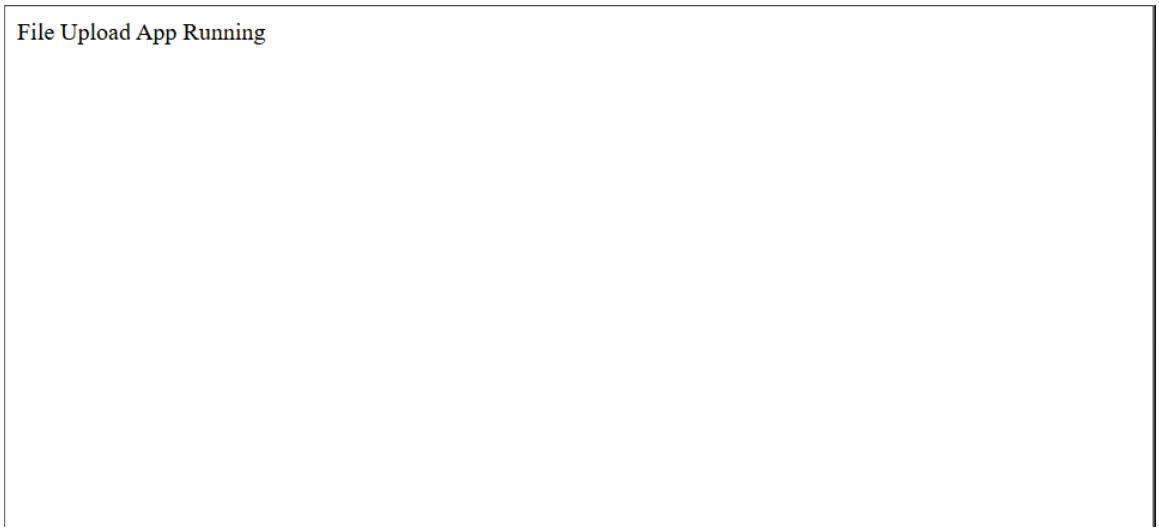
Explanation

- Validates file types
- Restricts file size
- Uses safe storage

Output

Safe upload only





Task 3 – API Security

Prompt
Generate API endpoint

AI Generated Code (Insecure)
```
from flask import Flask


app = Flask(__name__)


@app.route('/data')

def get_data():

  return {"message": "Public data"}


app.run()
```

Issues Identified
- No authentication
- Public access
- No security control

Secure Code (Fix)

```
from flask import Flask, request, jsonify


app = Flask(__name__)


API_KEY = "secure123"


def authenticate(req):
  return req.headers.get("API-Key") == API_KEY


@app.route('/data', methods=['GET'])
def get_data():
  if not authenticate(request):
    return jsonify({"error": "Unauthorized"}), 401


  return jsonify({"message": "Secure data"})


app.run()
```

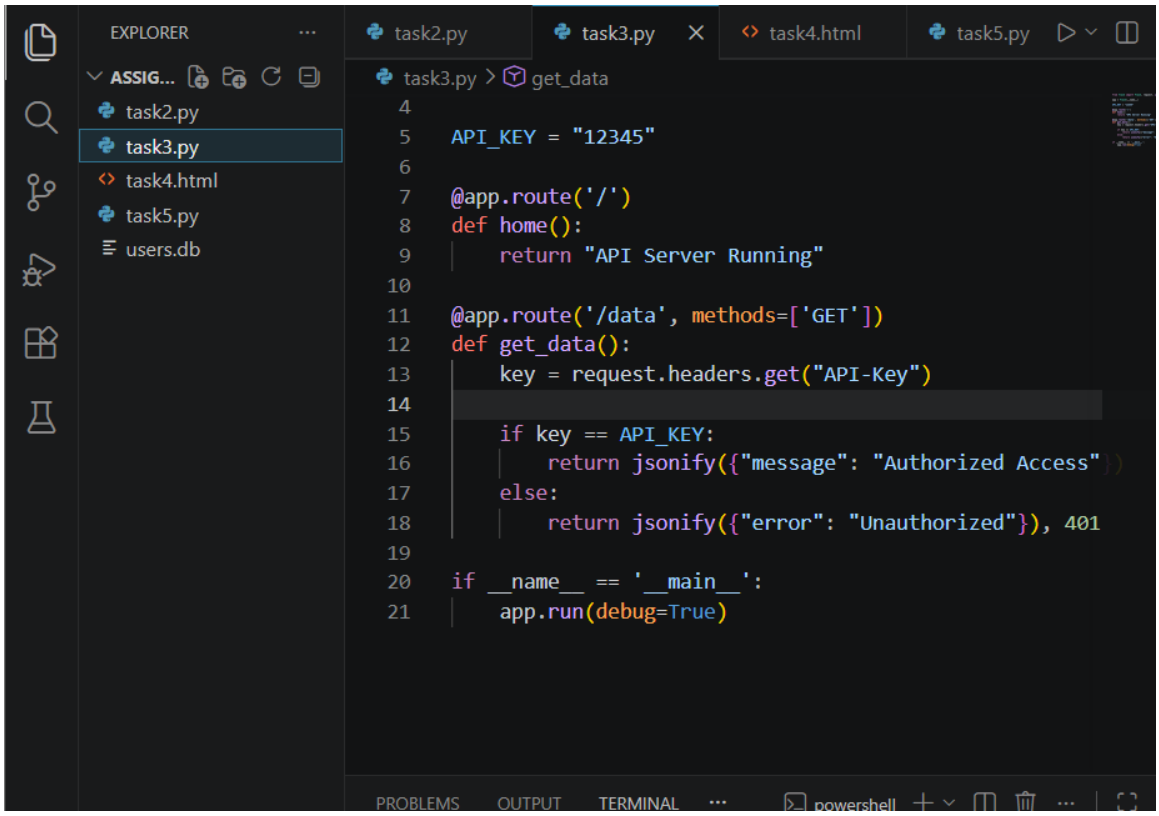
Explanation

 Adds API key authentication

 Restricts unauthorized users

Output

Secure response



API Server Running

Pretty-print


```
{
  "error": "Unauthorized"
}
```

Task 4 – XSS Prevention

Prompt

Generate feedback form

AI Generated Code (Insecure)

```
<input type="text" id="name">

<button onclick="display()">Submit</button>


<p id="output"></p>


<script>

function display() {

  var name = document.getElementById("name").value;

  document.getElementById("output").innerHTML = name;

}

</script>
```

Issues Identified

XSS vulnerability

 Uses innerHTML

 Allows XSS attacks

Secure Code (Fix)

```
textContent=userInput

<input type="text" id="name">

<button onclick="display()">Submit</button>


<p id="output"></p>


<script>
```

```
function display() {
   var name = document.getElementById("name").value;
   document.getElementById("output").textContent = name;
}
</script>
```

Explanation

Prevents script execution

Output

Safe display

Task 5 – SQL Injection

Prompt

Generate DB script

AI Generated Code (Insecure)

```
import sqlite3
```

```
conn = sqlite3.connect("users.db")

cursor = conn.cursor()


username = input("Enter username: ")


query = "SELECT * FROM users WHERE username = '" + username + "'"

cursor.execute(query)


print(cursor.fetchall())
```

Issues Identified

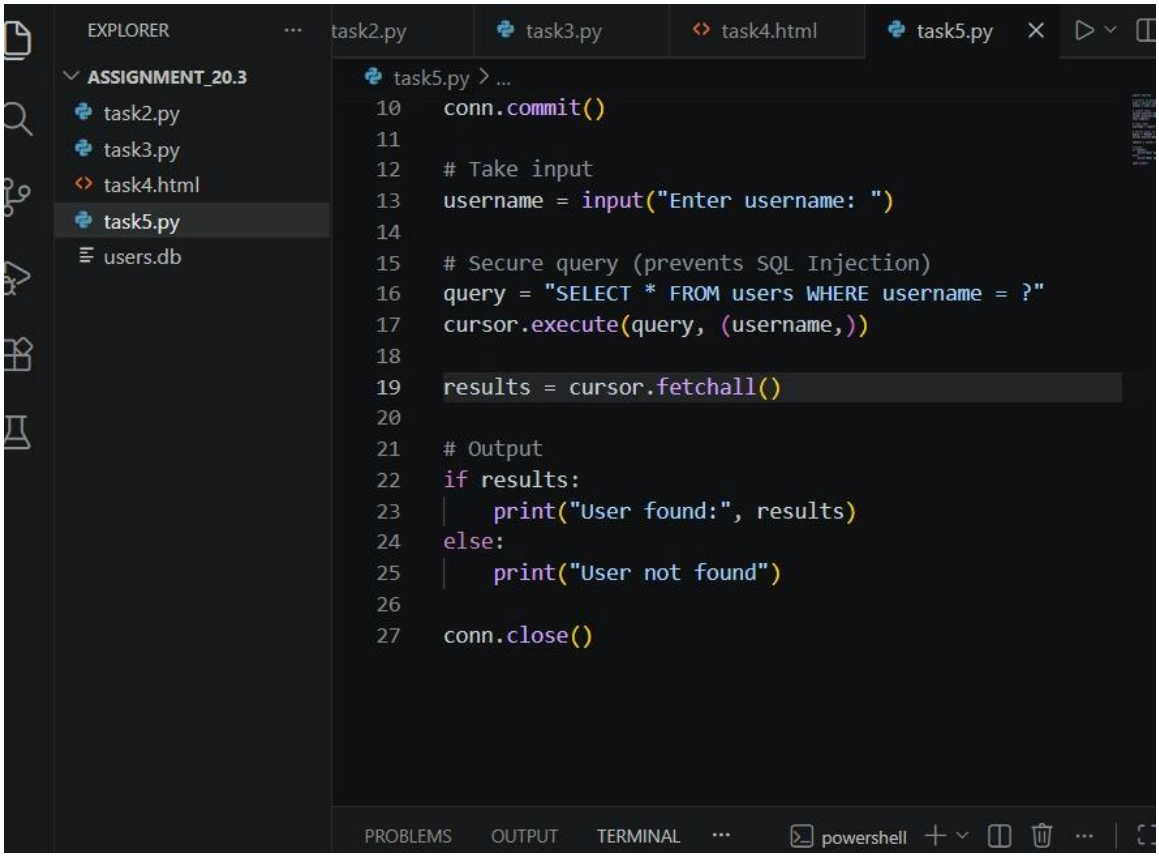
SQL Injection risk

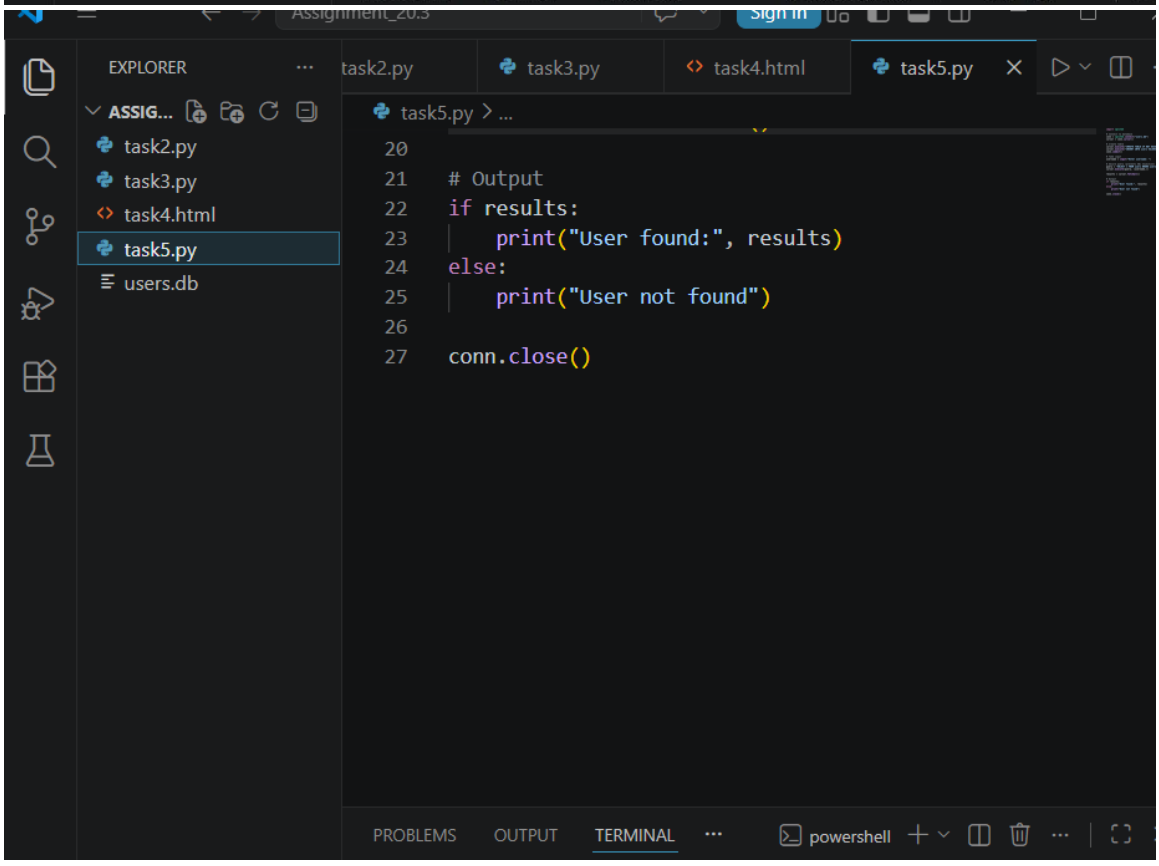
Secure Code (Fix)

```
import sqlite3


conn = sqlite3.connect("users.db")

cursor = conn.cursor()


username = input("Enter username: ")


query = "SELECT * FROM users WHERE username = ?"

cursor.execute(query, (username,))


print(cursor.fetchall())
```

Explanation

Prevents injection


EXPLORER
ASSIGNMENT_20.3
task2.py
task3.py
task4.html
task5.py
users.db
task2.py
task3.py
task4.html
task5.py
task5.py > ...
10 conn.commit()
11
12 # Take input
13 username = input("Enter username: ")
14
15 # Secure query (prevents SQL Injection)
16 query = "SELECT * FROM users WHERE username = ?"
17 cursor.execute(query, (username,))
18
19 results = cursor.fetchall()
20
21 # Output
22 if results:
23 print("User found:", results)
24 else:
25 print("User not found")
26
27 conn.close()
PROBLEMS
OUTPUT
TERMINAL
powershell


EXPLORER
ASSIG...
task2.py
task3.py
task4.html
task5.py
users.db
task2.py
task3.py
task4.html
task5.py
task5.py > ...
20
21 # Output
22 if results:
23 print("User found:", results)
24 else:
25 print("User not found")
26
27 conn.close()
PROBLEMS
OUTPUT
TERMINAL
powershell

Conclusion

AI-generated code may contain security vulnerabilities. By applying proper validation, authentication, and secure coding practices, we can improve application security, reliability, and performance